

Reinforcement Learning in Robotics: A Survey

Jens Kober^{*†}

J. Andrew Bagnell[‡]

Jan Peters^{§¶}

email: jkober@cor-lab.uni-bielefeld.de, dbagnell@ri.cmu.edu, mail@jan-peters.net

Reinforcement learning offers to robotics a framework and set of tools for the design of sophisticated and hard-to-engineer behaviors. Conversely, the challenges of robotic problems provide both inspiration, impact, and validation for developments in reinforcement learning. The relationship between disciplines has sufficient promise to be likened to that between physics and mathematics. In this article, we attempt to strengthen the links between the two research communities by providing a survey of work in reinforcement learning for behavior generation in robots. We highlight both key challenges in robot reinforcement learning as well as notable successes. We discuss how contributions tamed the complexity of the domain and study the role of algorithms, representations, and prior knowledge in achieving these successes. As a result, a particular focus of our paper lies on the choice between model-based and model-free as well as between value function-based and policy search methods. By analyzing a simple problem in some detail we demonstrate how reinforcement learning approaches may be profitably applied, and we note throughout open questions and the tremendous potential for future research.

keywords: reinforcement learning, learning control, robot, survey

1 Introduction

A remarkable variety of problems in robotics may be naturally phrased as ones of *reinforcement learning*. Reinforcement learning (RL) enables a robot to autonomously

^{*}Bielefeld University, CoR-Lab Research Institute for Cognition and Robotics, Universitätsstr. 25, 33615 Bielefeld, Germany

[†]Honda Research Institute Europe, Carl-Legien-Str. 30, 63073 Offenbach/Main, Germany

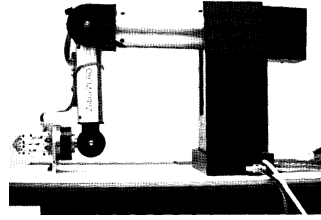
[‡]Carnegie Mellon University, Robotics Institute, 5000 Forbes Avenue, Pittsburgh, PA 15213, USA

[§]Max Planck Institute for Intelligent Systems, Department of Empirical Inference, Spemannstr. 38, 72076 Tübingen, Germany

[¶]Technische Universität Darmstadt, FB Informatik, FG Intelligent Autonomous Systems, Hochschulstr. 10, 64289 Darmstadt, Germany



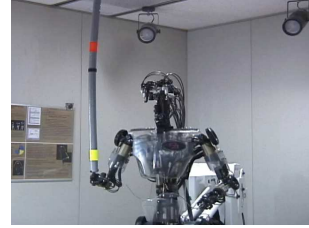
(a) OBELIX robot



(b) Zebra Zero robot



(c) Autonomous helicopter



(d) Sarcos humanoid DB

Figure 1: This figure illustrates a small sample of robots with behaviors that were reinforcement learned. These cover the whole range of aerial vehicles, robotic arms, autonomous vehicles, and humanoid robots. (a) The OBELIX robot is a wheeled mobile robot that learned to push boxes (Mahadevan and Connell, 1992) with a value function-based approach (Picture reprint with permission of Sridhar Mahadevan). (b) A Zebra Zero robot arm learned a peg-in-hole insertion task (Gullapalli et al., 1994) with a model-free policy gradient approach (Picture reprint with permission of Rod Grupen). (c) Carnegie Mellon’s autonomous helicopter leveraged a model-based policy search approach to learn a robust flight controller (Bagnell and Schneider, 2001). (d) The Sarcos humanoid DB learned a pole-balancing task (Schaal, 1996) using forward models (Picture reprint with permission of Stefan Schaal).

discover an optimal behavior through trial-and-error interactions with its environment. Instead of explicitly detailing the solution to a problem, in reinforcement learning the designer of a control task provides feedback in terms of a scalar objective function that measures the one-step performance of the robot. Figure 1 illustrates the diverse set of robots that have learned tasks using reinforcement learning.

Consider, for example, attempting to train a robot to return a table tennis ball over the net (Muelling et al., 2012). In this case, the robot might make an observations of dynamic variables specifying ball position and velocity and the internal dynamics of the joint position and velocity. This might in fact capture well the *state* s of the system – providing a complete statistic for predicting future observations. The *actions* a available to the robot might be the torque sent to motors or the desired accelerations sent to an inverse dynamics control system. A function π that generates the motor commands (i.e., the actions) based on the incoming ball and current internal arm observations (i.e.,

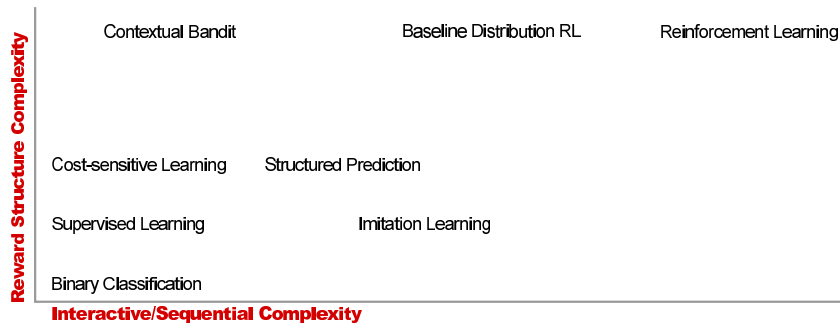


Figure 2: A illustration of the inter-relations between well-studied learning problems in the literature along axes that attempt to capture both the information and complexity available in reward signals and the complexity of sequential interaction between learner and environment. Each problem subsumes those to the left and below; reduction techniques provide methods whereby harder problems (above and right) may be addressed using repeated application of algorithms built for simpler problems. (Langford and Zadrozny, 2005)

the state) would be called the *policy*. A reinforcement learning problem is to find a policy that optimizes the long term sum of *rewards* $R(s, a)$; a reinforcement learning algorithm is one designed to find such a (near)-optimal policy. The reward function in this example could be based on the success of the hits as well as secondary criteria like energy consumption.

1.1 Reinforcement Learning in the Context of Machine Learning

In the problem of reinforcement learning, an agent explores the space of possible strategies and receives feedback on the outcome of the choices made. From this information, a “good” – or ideally optimal – policy (i.e., strategy or controller) must be deduced.

Reinforcement learning may be understood by contrasting the problem with other areas of study in machine learning. In supervised learning (Langford and Zadrozny, 2005), an agent is directly presented a sequence of independent examples of correct predictions to make in different circumstances. In imitation learning, an agent is provided demonstrations of actions of a good strategy to follow in given situations (Argall et al., 2009; Schaal, 1999).

To aid in understanding the RL problem and its relation with techniques widely used within robotics, Figure 2 provides a schematic illustration of two axes of problem variability: the complexity of sequential interaction and the complexity of reward structure. This hierarchy of problems, and the relations between them, is a complex one, varying in manifold attributes and difficult to condense to something like a simple linear ordering on problems. Much recent work in the machine learning community has focused on understanding the diversity and the inter-relations between problem classes. The figure should be understood in this light as providing a crude picture of the relationship between areas of machine learning research important for robotics.

Each problem subsumes those that are both below and to the left in the sense that one may always frame the simpler problem in terms of the more complex one; note that some problems are not linearly ordered. In this sense, reinforcement learning subsumes much of the scope of classical machine learning as well as contextual bandit and imitation learning problems. Reduction algorithms (Langford and Zadrozny, 2005) are used to convert effective solutions for one class of problems into effective solutions for others, and have proven to be a key technique in machine learning.

At lower left, we find the paradigmatic problem of supervised learning, which plays a crucial role in applications as diverse as face detection and spam filtering. In these problems (including binary classification and regression), a learner’s goal is to map observations (typically known as features or covariates) to actions which are usually a discrete set of classes or a real value. These problems possess no interactive component: the design and analysis of algorithms to address these problems rely on training and testing instances as independent and identical distributed random variables. This rules out any notion that a decision made by the learner will impact future observations: supervised learning algorithms are built to operate in a world in which every decision has no effect on the future examples considered. Further, within supervised learning scenarios, during a training phase the “correct” or preferred answer is provided to the learner, so there is no ambiguity about action choices.

More complex reward structures are also often studied: one such is known as cost-sensitive learning, where each training example and each action or prediction is annotated with a cost for making such a prediction. Learning techniques exist that reduce such problems to the simpler classification problem, and active research directly addresses such problems as they are crucial in practical learning applications.

Contextual bandit or associative reinforcement learning problems begin to address the fundamental problem of exploration-vs-exploitation, as information is provided only about a chosen action and not what-might-have-been. These find wide-spread application in problems as diverse as pharmaceutical drug discovery to ad placement on the web, and are one of the most active research areas in the field.

Problems of imitation learning and structured prediction may be seen to vary from supervised learning on the alternate dimension of sequential interaction. Structured prediction, a key technique used within computer vision and robotics, where many predictions are made in concert by leveraging inter-relations between them, may be seen as a simplified variant of imitation learning (Daumé III et al., 2009; Ross et al., 2011a). In imitation learning, we assume that an expert (for example, a human pilot) that we wish to mimic provides demonstrations of a task. While “correct answers” are provided to the learner, complexity arises because any mistake by the learner modifies the future observations from what would have been seen had the expert chosen the controls. Such problems provably lead to compounding errors and violate the basic assumption of independent examples required for successful supervised learning. In fact, in sharp contrast with supervised learning problems where only a single data-set needs to be collected, repeated interaction between learner and teacher appears to both necessary and sufficient (Ross et al., 2011b) to provide performance guarantees in both theory and practice in imitation learning problems.

Reinforcement learning embraces the full complexity of these problems by requiring both interactive, sequential prediction as in imitation learning as well as complex reward structures with only “bandit” style feedback on the actions actually chosen. It is this combination that enables so many problems of relevance to robotics to be framed in these terms; it is this same combination that makes the problem both information-theoretically and computationally hard.

We note here briefly the problem termed “Baseline Distribution RL”: this is the standard RL problem with the additional benefit for the learner that it may draw initial states from a distribution provided by an expert instead of simply an initial state chosen by the problem. As we describe further in Section 5.1, this additional information of which states matter dramatically affects the complexity of learning.

1.2 Reinforcement Learning in the Context of Optimal Control

Reinforcement Learning (RL) is very closely related to the theory of classical optimal control, as well as dynamic programming, stochastic programming, simulation-optimization, stochastic search, and optimal stopping (Powell, 2012). Both RL and optimal control address the problem of finding an optimal policy (often also called the controller or control policy) that optimizes an objective function (i.e., the accumulated cost or reward), and both rely on the notion of a system being described by an underlying set of states, controls and a plant or model that describes transitions between states. However, optimal control assumes perfect knowledge of the system’s description in the form of a model (i.e., a function T that describes what the next state of the robot will be given the current state and action). For such models, optimal control ensures strong guarantees which, nevertheless, often break down due to model and computational approximations. In contrast, reinforcement learning operates directly on measured data and rewards from interaction with the environment. Reinforcement learning research has placed great focus on addressing cases which are analytically intractable using approximations and data-driven techniques. One of the most important approaches to reinforcement learning within robotics centers on the use of classical optimal control techniques (e.g. Linear-Quadratic Regulation and Differential Dynamic Programming) to system models learned via repeated interaction with the environment (Atkeson, 1998; Bagnell and Schneider, 2001; Coates et al., 2009). A concise discussion of viewing reinforcement learning as “adaptive optimal control” is presented in (Sutton et al., 1991).

1.3 Reinforcement Learning in the Context of Robotics

Robotics as a reinforcement learning domain differs considerably from most well-studied reinforcement learning benchmark problems. In this article, we highlight the challenges faced in tackling these problems. Problems in robotics are often best represented with high-dimensional, continuous states and actions (note that the 10-30 dimensional continuous actions common in robot reinforcement learning are considered large (Powell, 2012)). In robotics, it is often unrealistic to assume that the true state is completely observable and noise-free. The learning system will not be able to know precisely in

which state it is and even vastly different states might look very similar. Thus, robotics reinforcement learning are often modeled as partially observed, a point we take up in detail in our formal model description below. The learning system must hence use filters to estimate the true state. It is often essential to maintain the information state of the environment that not only contains the raw observations but also a notion of uncertainty on its estimates (e.g., both the mean and the variance of a Kalman filter tracking the ball in the robot table tennis example).

Experience on a real physical system is tedious to obtain, expensive and often hard to reproduce. Even getting to the same initial state is impossible for the robot table tennis system. Every single trial run, also called a roll-out, is costly and, as a result, such applications force us to focus on difficulties that do not arise as frequently in classical reinforcement learning benchmark examples. In order to learn within a reasonable time frame, suitable approximations of state, policy, value function, and/or system dynamics need to be introduced. However, while real-world experience is costly, it usually cannot be replaced by learning in simulations alone. In analytical or learned models of the system even small modeling errors can accumulate to a substantially different behavior, at least for highly dynamic tasks. Hence, algorithms need to be robust with respect to models that do not capture all the details of the real system, also referred to as *under-modeling*, and to model uncertainty. Another challenge commonly faced in robot reinforcement learning is the generation of appropriate reward functions. Rewards that guide the learning system quickly to success are needed to cope with the cost of real-world experience. This problem is called *reward shaping* (Laud, 2004) and represents a substantial manual contribution. Specifying good reward functions in robotics requires a fair amount of domain knowledge and may often be hard in practice.

Not every reinforcement learning method is equally suitable for the robotics domain. In fact, many of the methods thus far demonstrated on difficult problems have been model-based (Atkeson et al., 1997; Abbeel et al., 2007; Deisenroth and Rasmussen, 2011) and robot learning systems often employ policy search methods rather than value function-based approaches (Gullapalli et al., 1994; Miyamoto et al., 1996; Bagnell and Schneider, 2001; Kohl and Stone, 2004; Tedrake et al., 2005; Peters and Schaal, 2008a,b; Kober and Peters, 2009; Deisenroth et al., 2011). Such design choices stand in contrast to possibly the bulk of the early research in the machine learning community (Kaelbling et al., 1996; Sutton and Barto, 1998). We attempt to give a fairly complete overview on real robot reinforcement learning citing most original papers while grouping them based on the key insights employed to make the Robot Reinforcement Learning problem tractable. We isolate key insights such as choosing an appropriate representation for a value function or policy, incorporating prior knowledge, and transfer knowledge from simulations.

This paper surveys a wide variety of tasks where reinforcement learning has been successfully applied to robotics. If a task can be phrased as an optimization problem and exhibits temporal structure, reinforcement learning can often be profitably applied to both phrase and solve that problem. The goal of this paper is twofold. On the one hand, we hope that this paper can provide indications for the robotics community which type of problems can be tackled by reinforcement learning and provide pointers to approaches that are promising. On the other hand, for the reinforcement learning community, this

paper can point out novel real-world test beds and remarkable opportunities for research on open questions. We focus mainly on results that were obtained on physical robots with tasks going beyond typical reinforcement learning benchmarks.

We concisely present reinforcement learning techniques in the context of robotics in Section 2. The challenges in applying reinforcement learning in robotics are discussed in Section 3. Different approaches to making reinforcement learning tractable are treated in Sections 4 to 6. In Section 7, the example of ball-in-a-cup is employed to highlight which of the various approaches discussed in the paper have been particularly helpful to make such a complex task tractable. Finally, in Section 8, we summarize the specific problems and benefits of reinforcement learning in robotics and provide concluding thoughts on the problems and promise of reinforcement learning in robotics.

2 A Concise Introduction to Reinforcement Learning

In reinforcement learning, an agent tries to maximize the accumulated reward over its life-time. In an *episodic* setting, where the task is restarted after each end of an episode, the objective is to maximize the total reward per episode. If the task is on-going without a clear beginning and end, either the average reward over the whole life-time or a discounted return (i.e., a weighted average where distant rewards have less influence) can be optimized. In such reinforcement learning problems, the agent and its environment may be modeled being in a state $s \in S$ and can perform actions $a \in A$, each of which may be members of either discrete or continuous sets and can be multi-dimensional. A state s contains all relevant information about the current situation to predict future states (or observables); an example would be the current position of a robot in a navigation task¹. An action a is used to control (or change) the state of the system. For example, in the navigation task we could have the actions corresponding to torques applied to the wheels. For every step, the agent also gets a reward R , which is a scalar value and assumed to be a function of the state and observation. (It may equally be modeled as a random variable that depends on only these variables.) In the navigation task, a possible reward could be designed based on the energy costs for taken actions and rewards for reaching targets. The goal of reinforcement learning is to find a mapping from states to actions, called policy π , that picks actions a in given states s maximizing the cumulative expected reward. The policy π is either deterministic or probabilistic. The former always uses the exact same action for a given state in the form $a = \pi(s)$, the later draws a sample from a distribution over actions when it encounters a state, i.e., $a \sim \pi(s, a) = P(a|s)$. The reinforcement learning agent needs to discover the relations between states, actions, and rewards. Hence exploration is required which can either be directly embedded in the policy or performed separately and only as part of the learning process.

Classical reinforcement learning approaches are based on the assumption that we have a *Markov Decision Process* (MDP) consisting of the set of states S , set of actions A , the rewards R and transition probabilities T that capture the dynamics of a system.

¹When only observations but not the complete state is available, the sufficient statistics of the filter can alternatively serve as state s . Such a state is often called information or belief state.

ing algorithm does not exploit model inaccuracies. See Section 6 for a more detailed discussion.

3 Challenges in Robot Reinforcement Learning

Reinforcement learning is generally a hard problem and many of its challenges are particularly apparent in the robotics setting. As the states and actions of most robots are inherently continuous, we are forced to consider the resolution at which they are represented. We must decide how fine grained the control is that we require over the robot, whether we employ discretization or function approximation, and what time step we establish. Additionally, as the dimensionality of both states and actions can be high, we face the “Curse of Dimensionality” (Bellman, 1957) as discussed in Section 3.1. As robotics deals with complex physical systems, samples can be expensive due to the long execution time of complete tasks, required manual interventions, and the need maintenance and repair. In these real-world measurements, we must cope with the uncertainty inherent in complex physical systems. A robot requires that the algorithm runs in real-time. The algorithm must be capable of dealing with delays in sensing and execution that are inherent in physical systems (see Section 3.2). A simulation might alleviate many problems but these approaches need to be robust with respect to model errors as discussed in Section 3.3. An often underestimated problem is the goal specification, which is achieved by designing a good reward function. As noted in Section 3.4, this choice can make the difference between feasibility and an unreasonable amount of exploration.

3.1 Curse of Dimensionality

When Bellman (1957) explored optimal control in discrete high-dimensional spaces, he faced an exponential explosion of states and actions for which he coined the term “Curse of Dimensionality”. As the number of dimensions grows, exponentially more data and computation are needed to cover the complete state-action space. . For example, if we assume that each dimension of a state-space is discretized into ten levels, we have 10 states for a one-dimensional state-space, $10^3 = 1000$ unique states for a three-dimensional state-space, and 10^n possible states for a n -dimensional state space. Evaluating every state quickly becomes infeasible with growing dimensionality, even for discrete states. Bellman originally coined the term in the context of optimization, but it also applies to function approximation and numerical integration (Donoho, 2000). While supervised learning methods have tamed this exponential growth by considering only competitive optimality with respect to a limited class of function approximators, such results are much more difficult in reinforcement learning where data must be collected throughout state-space to ensure global optimality.

Robotic systems often have to deal with these high dimensional states and actions due to the many degrees of freedom of modern anthropomorphic robots. For example, in the ball-paddling task shown in Figure 3, a proper representation of a robot’s state would consist of its joint angles and velocities for each of its seven degrees of freedom as well as the Cartesian position and velocity of the ball. The robot’s actions would be the

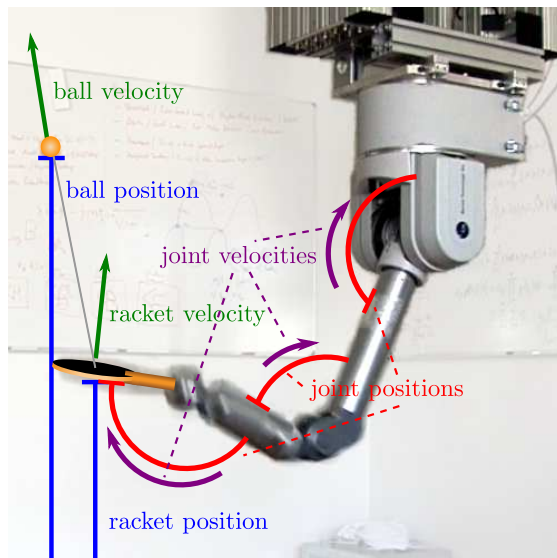


Figure 3: This Figure illustrates the state space used in the modeling of a robot reinforcement learning task of paddling a ball.

generated motor commands, which often are torques or accelerations. In this example, we have $2 \times (7 + 3) = 20$ state dimensions and 7-dimensional continuous actions. Obviously, other tasks may require even more dimensions. For example, human-like actuation often follows the antagonistic principle (Yamaguchi and Takanishi, 1997) which additionally enables control of stiffness. Such dimensionality is a major challenge for both the robotics and the reinforcement learning communities.

In robotics, such tasks are often rendered tractable to the robot engineer by a hierarchical task decomposition that shifts some complexity to a lower layer of functionality. Classical reinforcement learning approaches often consider a grid-based representation with discrete states and actions, often referred to as a *grid-world*. A navigational task for mobile robots could be projected into this representation by employing a number of actions like “move to the cell to the left” that use a lower level controller that takes care of accelerating, moving, and stopping while ensuring precision. In the ball-paddling example, we may simplify by controlling the robot in racket space (which is lower-dimensional as the racket is orientation-invariant around the string’s mounting point) with an operational space control law (Nakanishi et al., 2008). Many commercial robot systems also encapsulate some of the state and action components in an embedded control system (e.g., trajectory fragments are frequently used as actions for industrial robots). However, this form of a state dimensionality reduction severely limits the dynamic capabilities of the robot according to our experience (Schaal et al., 2002; Peters et al., 2010b).

The reinforcement learning community has a long history of dealing with dimensionality using computational abstractions. It offers a larger set of applicable tools ranging from adaptive discretizations (Buşoniu et al., 2010) and function approximation approaches (Sutton and Barto, 1998) to macro-actions or options (Barto and Mahadevan, 2003; Hart

and Grupen, 2011). Options allow a task to be decomposed into elementary components and quite naturally translate to robotics. Such options can autonomously achieve a sub-task, such as opening a door, which reduces the planning horizon (Barto and Mahadevan, 2003). The automatic generation of such sets of options is a key issue in order to enable such approaches. We will discuss approaches that have been successful in robot reinforcement learning in Section 4.

3.2 Curse of Real-World Samples

Robots inherently interact with the physical world. Hence, robot reinforcement learning suffers from most of the resulting real-world problems. For example, robot hardware is usually expensive, suffers from wear and tear, and requires careful maintenance. Repairing a robot system is a non-negligible effort associated with cost, physical labor and long waiting periods. To apply reinforcement learning in robotics, safe exploration becomes a key issue of the learning process (Schneider, 1996; Bagnell, 2004; Deisenroth and Rasmussen, 2011; Moldovan and Abbeel, 2012), a problem often neglected in the general reinforcement learning community. Perkins and Barto (2002) have come up with a method for constructing reinforcement learning agents based on Lyapunov functions. Switching between the underlying controllers is always safe and offers basic performance guarantees.

However, several more aspects of the real-world make robotics a challenging domain. As the dynamics of a robot can change due to many external factors ranging from temperature to wear, the learning process may never fully converge, i.e., it needs a “tracking solution” (Sutton et al., 2007). Frequently, the environment settings during an earlier learning period cannot be reproduced. External factors are not always clear – for example, how light conditions affect the performance of the vision system and, as a result, the task’s performance. This problem makes comparing algorithms particularly hard. Furthermore, the approaches often have to deal with uncertainty due to inherent measurement noise and the inability to observe all states directly with sensors.

Most real robot learning tasks require some form of human supervision, e.g., putting the pole back on the robot’s end-effector during pole balancing (see Figure 1d) after a failure. Even when an automatic reset exists (e.g., by having a smart mechanism that resets the pole), learning speed becomes essential as a task on a real robot cannot be sped up. In some tasks like a slowly rolling robot, the dynamics can be ignored; in others like a flying robot, they cannot. Especially in the latter case, often the whole episode needs to be completed as it is not possible to start from arbitrary states.

For such reasons, real-world samples are expensive in terms of time, labor and, potentially, finances. In robotic reinforcement learning, it is often considered to be more important to limit the real-world interaction time instead of limiting memory consumption or computational complexity. Thus, sample efficient algorithms that are able to learn from a small number of trials are essential. In Section 6 we will point out several approaches that allow the amount of required real-world interactions to be reduced.

Since the robot is a physical system, there are strict constraints on the interaction between the learning algorithm and the robot setup. For dynamic tasks, the movement

cannot be paused and actions must be selected within a time-budget without the opportunity to pause to think, learn or plan between actions. These constraints are less severe in an episodic setting where the time intensive part of the learning can be postponed to the period between episodes. Hester et al. (2012) has proposed a real-time architecture for model-based value function reinforcement learning methods taking into account these challenges.

As reinforcement learning algorithms are inherently implemented on a digital computer, the discretization of time is unavoidable despite that physical systems are inherently continuous time systems. Time-discretization of the actuation can generate undesirable artifacts (e.g., the distortion of distance between states) even for idealized physical systems, which cannot be avoided. As most robots are controlled at fixed sampling frequencies (in the range between 500Hz and 3kHz) determined by the manufacturer of the robot, the upper bound on the rate of temporal discretization is usually pre-determined. The lower bound depends on the horizon of the problem, the achievable speed of changes in the state, as well as delays in sensing and actuation.

All physical systems exhibit such delays in sensing and actuation. The state of the setup (represented by the filtered sensor signals) may frequently lag behind the real state due to processing and communication delays. More critically, there are also communication delays in actuation as well as delays due to the fact that neither motors, gear boxes nor the body’s movement can change instantly. Due to these delays, actions may not have instantaneous effects but are observable only several time steps later. In contrast, in most general reinforcement learning algorithms, the actions are assumed to take effect instantaneously as such delays would violate the usual Markov assumption. This effect can be addressed by putting some number of recent actions into the state. However, this significantly increases the dimensionality of the problem.

The problems related to time-budgets and delays can also be avoided by increasing the duration of the time steps. One downside of this approach is that the robot cannot be controlled as precisely; another is that it may complicate a description of system dynamics.

3.3 Curse of Under-Modeling and Model Uncertainty

One way to offset the cost of real-world interaction is to use accurate models as simulators. In an ideal setting, this approach would render it possible to learn the behavior in simulation and subsequently transfer it to the real robot. Unfortunately, creating a sufficiently accurate model of the robot and its environment is challenging and often requires very many data samples. As small model errors due to this under-modeling accumulate, the simulated robot can quickly diverge from the real-world system. When a policy is trained using an imprecise forward model as simulator, the behavior will not transfer without significant modifications as experienced by Atkeson (1994) when learning the underactuated pendulum swing-up. The authors have achieved a direct transfer in only a limited number of experiments; see Section 6.1 for examples.

For tasks where the system is self-stabilizing (that is, where the robot does not require active control to remain in a safe state or return to it), transferring policies often works

well. Such tasks often feature some type of dampening that absorbs the energy introduced by perturbations or control inaccuracies. If the task is inherently stable, it is safer to assume that approaches that were applied in simulation work similarly in the real world (Kober and Peters, 2010). Nevertheless, tasks can often be learned better in the real world than in simulation due to complex mechanical interactions (including contacts and friction) that have proven difficult to model accurately. For example, in the ball-paddling task (Figure 3) the elastic string that attaches the ball to the racket always pulls back the ball towards the racket even when hit very hard. Initial simulations (including friction models, restitution models, dampening models, models for the elastic string, and air drag) of the ball-racket contacts indicated that these factors would be very hard to control. In a real experiment, however, the reflections of the ball on the racket proved to be less critical than in simulation and the stabilizing forces due to the elastic string were sufficient to render the whole system self-stabilizing.

In contrast, in unstable tasks small variations have drastic consequences. For example, in a pole balancing task, the equilibrium of the upright pole is very brittle and constant control is required to stabilize the system. Transferred policies often perform poorly in this setting. Nevertheless, approximate models serve a number of key roles which we discuss in Section 6, including verifying and testing the algorithms in simulation, establishing proximity to theoretically optimal solutions, calculating approximate gradients for local policy improvement, identifying strategies for collecting more data, and performing “mental rehearsal”.

3.4 Curse of Goal Specification

In reinforcement learning, the desired behavior is implicitly specified by the reward function. The goal of reinforcement learning algorithms then is to maximize the accumulated long-term reward. While often dramatically simpler than specifying the behavior itself, in practice, it can be surprisingly difficult to define a good reward function in robot reinforcement learning. The learner must observe variance in the reward signal in order to be able to improve a policy: if the same return is always received, there is no way to determine which policy is better or closer to the optimum.

In many domains, it seems natural to provide rewards only upon task achievement – for example, when a table tennis robot wins a match. This view results in an apparently simple, binary reward specification. However, a robot may receive such a reward so rarely that it is unlikely to ever succeed in the lifetime of a real-world system. Instead of relying on simpler binary rewards, we frequently need to include intermediate rewards in the scalar reward function to guide the learning process to a reasonable solution, a process known as *reward shaping* (Laud, 2004).

Beyond the need to shorten the effective problem horizon by providing intermediate rewards, the trade-off between different factors may be essential. For instance, hitting a table tennis ball very hard may result in a high score but is likely to damage a robot or shorten its life span. Similarly, changes in actions may be penalized to avoid high frequency controls that are likely to be very poorly captured with tractable low dimensional state-space or rigid-body models. Reinforcement learning algorithms are also notorious

for exploiting the reward function in ways that are not anticipated by the designer. For example, if the distance between the ball and the desired highest point is part of the reward in ball paddling (see Figure 3), many locally optimal solutions would attempt to simply move the racket upwards and keep the ball on it. Reward shaping gives the system a notion of closeness to the desired behavior instead of relying on a reward that only encodes success or failure (Ng et al., 1999).

Often the desired behavior can be most naturally represented with a reward function in a particular state and action space. However, this representation does not necessarily correspond to the space where the actual learning needs to be performed due to both computational and statistical limitations. Employing methods to render the learning problem tractable often result in different, more abstract state and action spaces which might not allow accurate representation of the original reward function. In such cases, a reward artfully specified in terms of the features of the space in which the learning algorithm operates can prove remarkably effective. There is also a trade-off between the complexity of the reward function and the complexity of the learning problem. For example, in the ball-in-a-cup task (Section 7) the most natural reward would be a binary value depending on whether the ball is in the cup or not. To render the learning problem tractable, a less intuitive reward needed to be devised in terms of a Cartesian distance with additional directional information (see Section 7.1 for details). Another example is Crusher (Ratliff et al., 2006a), an outdoor robot, where the human designer was interested in a combination of minimizing time and risk to the robot. However, the robot reasons about the world on the long time horizon scale as if it was a very simple, deterministic, holonomic robot operating on a fine grid of continuous costs. Hence, the desired behavior cannot be represented straightforwardly in this state-space. Nevertheless, a remarkably human-like behavior that seems to respect time and risk priorities can be achieved by carefully mapping features describing each state (discrete grid location with features computed by an on-board perception system) to cost.

Inverse optimal control, also known as inverse reinforcement learning (Russell, 1998), is a promising alternative to specifying the reward function manually. It assumes that a reward function can be reconstructed from a set of expert demonstrations. This reward function does not necessarily correspond to the true reward function, but provides guarantees on the resulting performance of learned behaviors (Abbeel and Ng, 2004; Ratliff et al., 2006b). Inverse optimal control was initially studied in the control community (Kalman, 1964) and in the field of economics (Keeney and Raiffa, 1976). The initial results were only applicable to limited domains (linear quadratic regulator problems) and required closed form access to plant and controller, hence samples from human demonstrations could not be used. Russell (1998) brought the field to the attention of the machine learning community. Abbeel and Ng (2004) defined an important constraint on the solution to the inverse RL problem when reward functions are linear in a set of features: a policy that is extracted by observing demonstrations has to earn the same reward as the policy that is being demonstrated. Ratliff et al. (2006b) demonstrated that inverse optimal control can be understood as a generalization of ideas in machine learning of *structured prediction* and introduced efficient sub-gradient based algorithms with regret bounds that enabled large scale application of the technique within robotics. Ziebart

et al. (2008) extended the technique developed by Abbeel and Ng (2004) by rendering the idea robust and probabilistic, enabling its effective use for both learning policies and predicting the behavior of sub-optimal agents. These techniques, and many variants, have been recently successfully applied to outdoor robot navigation (Ratliff et al., 2006a; Silver et al., 2008, 2010), manipulation (Ratliff et al., 2007), and quadruped locomotion (Ratliff et al., 2006a, 2007; Kolter et al., 2007).

More recently, the notion that complex policies can be built on top of simple, easily solved optimal control problems by exploiting rich, parametrized reward functions has been exploited within reinforcement learning more directly. In (Sorg et al., 2010; Zucker and Bagnell, 2012), complex policies are derived by adapting a reward function for simple optimal control problems using policy search techniques. Zucker and Bagnell (2012) demonstrate that this technique can enable efficient solutions to robotic marble-maze problems that effectively transfer between mazes of varying design and complexity. These works highlight the natural trade-off between the complexity of the reward function and the complexity of the underlying reinforcement learning problem for achieving a desired behavior.

4 Tractability Through Representation

As discussed above, reinforcement learning provides a framework for a remarkable variety of problems of significance to both robotics and machine learning. However, the computational and information-theoretic consequences that we outlined above accompany this power and generality. As a result, naive application of reinforcement learning techniques in robotics is likely to be doomed to failure. The remarkable successes that we reference in this article have been achieved by leveraging a few key principles – effective representations, approximate models, and prior knowledge or information. In the following three sections, we review these principles and summarize how each has been made effective in practice. We hope that understanding these broad approaches will lead to new successes in robotic reinforcement learning by combining successful methods and encourage research on novel techniques that embody each of these principles.

Much of the success of reinforcement learning methods has been due to the clever use of approximate representations. The need of such approximations is particularly pronounced in robotics, where table-based representations (as discussed in Section 2.2.1) are rarely scalable. The different ways of making reinforcement learning methods tractable in robotics are tightly coupled to the underlying optimization framework. Reducing the dimensionality of states or actions by smart state-action discretization is a representational simplification that may enhance both policy search and value function-based methods (see 4.1). A value function-based approach requires an accurate and robust but general function approximator that can capture the value function with sufficient precision (see Section 4.2) while maintaining stability during learning. Policy search methods require a choice of policy representation that controls the complexity of representable policies to enhance learning speed (see Section 4.3). An overview of publications that make particular use of efficient representations to render the learning problem tractable is presented